Remote Sensing and Satellite Imagery

AI-powered Cloud Masking
Project Report

Team 1

Team members

Name	IDs	Contribution
Ibrahim Tarek Abdelazeem Amin	9210014	Classical ML
Mahmoud Sobhy Ramadan	9211090	Utils - Inference
Youssef Mohamed Rabie	9211430	U-Net - Data Processing
Youssef Mohamed Hagag	9211436	

Table Of Contents

Problem Understanding

We started by analysing the core problem: identifying and segmenting cloud regions from multi-band satellite images (TIFF format). These images contain four channels (Red, Green, Blue, and Infrared), and each has a corresponding binary mask indicating cloud locations. The goal was to develop a system capable of performing cloud segmentation with high accuracy and robustness under various conditions.

Literature Review

Given our limited timeline, we recognized early on that investing time in thorough research would significantly reduce development time. Therefore, we dedicated a focused effort to reviewing existing methods for satellite image segmentation and cloud masking.
We explored both traditional machine learning and modern deep learning approaches:

- Machine Learning Models – Random Forests

  Based on our review of related work, Random Forests emerged as a simple yet effective method for cloud detection. Its interpretability and relatively low computational requirements made it a strong candidate for initial prototyping. We also tried using an SVM (Support Vector Machine) model to explore other machine learning options, but it took a very long time to train, so we decided to cancel it.

- Deep Learning Models – U-Net

  We ultimately chose U-Net as our primary architecture for cloud segmentation. This decision was informed not only by its inclusion in our coursework :) but also by extensive literature support.

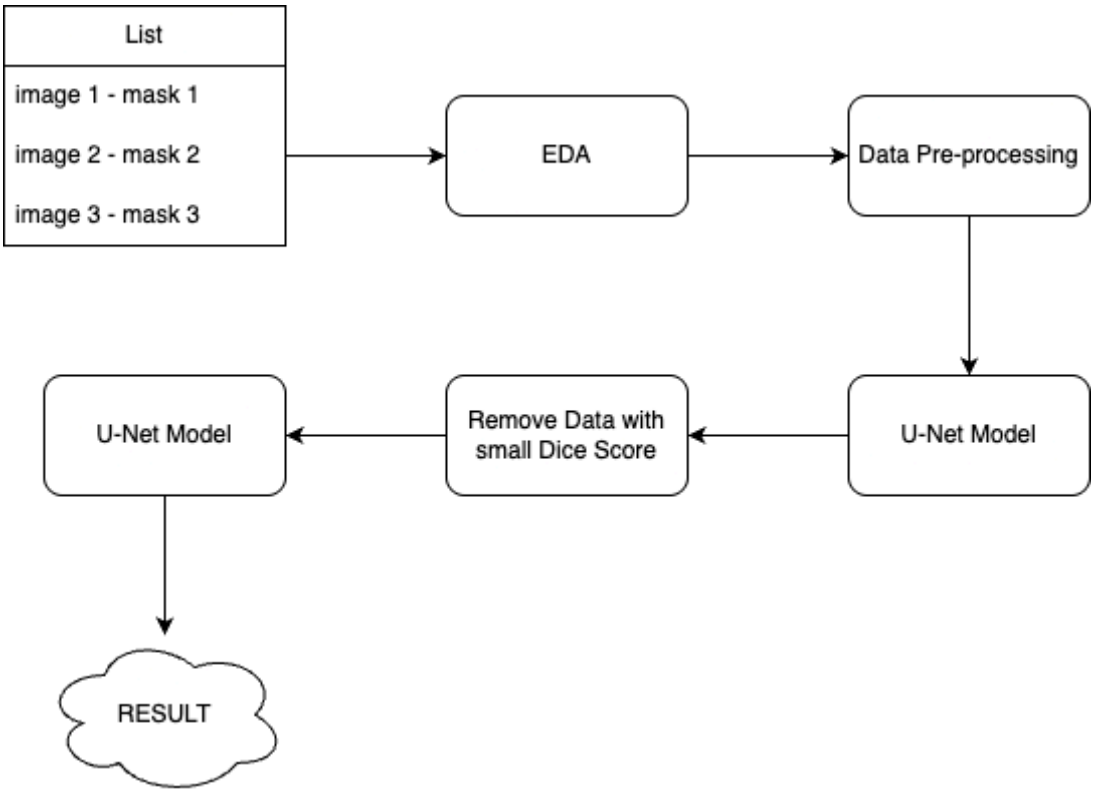
- Our research included the following resources:
    - Semantic Segmentation on SWIMSEG – PapersWithCode
    - Cloud Image Segmentation using U-Net Architecture in PyTorch

      A practical implementation of U-Net applied to satellite images, reinforcing its relevance to our task.
    - ACLNet: An Attention and Clustering-based Cloud Segmentation Network

      Although ACLNet introduces improvements over U-Net using attention mechanisms, we prioritized U-Net for its simplicity and proven effectiveness in our specific context.

In the end, we decided to use U-Net because it is proven to work well and is widely used for this type of task.

Project Pipeline



Dataset Preparation and Exploration

We worked with a dataset of around 10,000 TIFF images and their masks. Here's what we did:

- **Class Distribution**

  We checked how much cloud is in each image. We grouped them into 4 types:

  - No clouds (less than 1%)
  - Partial clouds (1% to 50%)
  - Heavy clouds (50% to 90%)
  - Full cloud cover (more than 90%)

    This helped us understand how different the images are and what the model needs to learn.

- **Checking the Data**

  We looked at some random images and their masks. This helped us notice that some masks were wrong or not accurate.

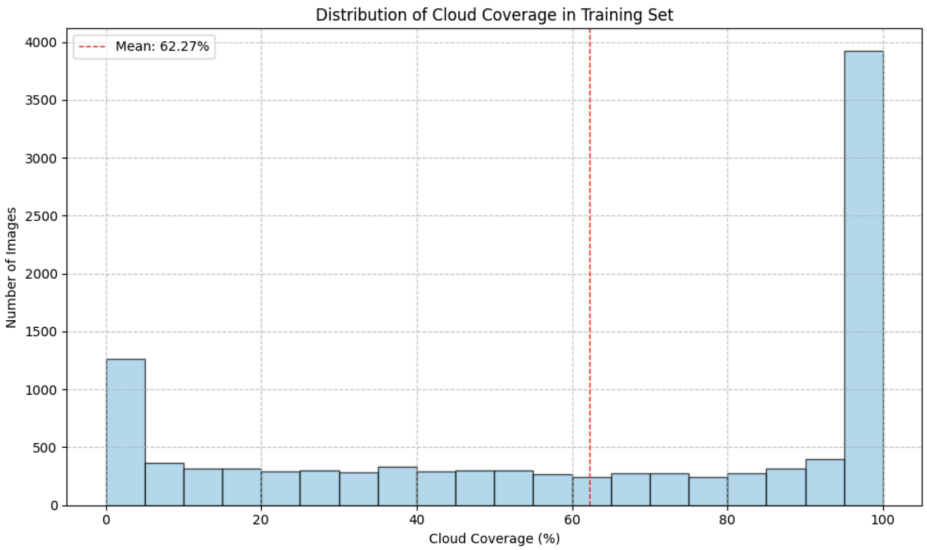
Cloud Coverage Categories:

No clouds (<1%):
858 images (8.12%)

Partial clouds (1-55%):
3491 images (33.02%)

Heavy clouds (55-99%):
2901 images (27.44%)

Full cloud cover (>99%):
3323 images (31.43%)



Preprocessing

Before training, we applied several steps to prepare the satellite images and their cloud masks:

1. **Data Augmentation and Normalization**

   To make the model learn better, we applied these random changes to training images:

   - Resize to a fixed size
   - Rotate up to ± 35° (50% chance)
   - Flip horizontally (50% chance)
   - Flip vertically (10% chance)
   - Scale pixel values from 0–255 to 0–1
   - Convert to PyTorch tensors

For validation, we only resized and scaled the images (no random changes).

3. **Data Splitting and Loading**

   We split the data into 80% training and 20% validation. Then we created custom PyTorch datasets and used data loaders with:

   - Batch size = 16
   - Shuffling for training
   - Multiple workers for faster loading
   - Pinned memory for better GPU performance

These steps prepared the data for efficient training of the segmentation model.

At first, we thought about going through each image and its mask manually, but this would take too much time and wouldn't add much value. So, we decided not to do that now. Instead, we'll explain our better solution for handling wrong labels later in the report.

Model Selection and Training

• Classical Machine Learning Model: Random Forest

We chose the Random Forest algorithm. However, it's not GPU-friendly, so it runs on the CPU only. Due to RAM limitations, we trained it on just 4,000 images and sampled 2,000 pixels from each image. Using more would cause memory crashes.

For hyperparameters, we used:

- n_estimators = 20
- random_state = 42
- n_jobs = -1 (to use all CPU cores)

We know that GridSearchCV could help find better hyperparameters, but it would take a very long time. So instead, we manually tried a few values and went with what worked reasonably well.

• Deep Learning Model: U-Net

We built a customized U-Net architecture with:

- **4 input channels** (Red, Green, Blue, Infrared)
- **Batch Normalization** to stabilize training
- **Dropout** to reduce overfitting

The U-Net consists of:

- A downsampling path (encoder)
- A bottleneck layer
- An upsampling path (decoder)
- Skip connections to preserve spatial information

• Loss Function

We used a combination of **Binary Cross-Entropy (BCE)** and **Dice Loss** to balance precision and recall. However, using Dice Loss alone gave very poor results, so we relied mostly on the combined loss.

• Training Setup

We trained the U-Net using:

- The Adam optimizer
- Mini-batches with a size of 16 (32 not possible due to RAM limitation)
- Image augmentations during training
- GPU acceleration to speed up training

Evaluation and Error Analysis

	U-Net	Random Forest
Dice Score	92.75% then 96.63%	77.07%
Size	124.2 MB	2315.74 MB

We trained the first U-Net model, then removed samples with low Dice scores (poor-quality data) and retrained the model on the cleaned dataset.

Model Profiling

U-Net	Random Forest
Parameters: 31,038,208 Operations: 13998.22 GFLOPs	File Size: 2315.74 MB Number of Trees: 20 Total Nodes (All Trees): 33725258 Average Tree Depth: 59.15 Approximate Operations per Prediction: 1183

Enhancements and Future Work

Our project works well, but there are still some ways to make it better in the future:

1. Use of ACLNet

In future work, we can try using ACLNet instead of other models. ACLNet is a powerful deep learning model that has ranked first in many online competitions like our project.

2. Improve Random Forest Model

Right now, our Random Forest model uses default settings. To make it better, we can use a method called GridSearchCV. This helps us find the best values for important settings like:

- max_depth (the depth of each tree),
- min_samples_split (the minimum number of samples required to split a node)
- n_estimators (the number of trees in the forest).